

TE 1



Java IO

Package

Y a 2 package IO :

- [java.io](#) -> classic io package
- [java.nio](#) -> moderne package depuis java 1.4, plus efficace et flexible mais plus complexe à prendre en main

Data type

Y a 2 type de données :

- Binary : c'est les bytes[], donc la donner en brute
- Text : c'est une version interprétée de la donnée pour la transformer en texte

Utiliser IO

En gros, y a InputStream and OutputStream, c'est des Streams (no way) et ça a le même fonctionnement / logique qu'en c++

- Open the stream
- Use it
- Close the stream

Pour les bytes[]

Exemple Output (pour écrire sur un fichier) stream (**FOR BYTES[]**) :

```

OutputStream fs = new FileOutputStream("file.data");

BufferedOutputStream bos = new BufferedOutputStream(fs);

for (int i = 0; i < 256; i++) {
    bos.write(i);
}

// flush le buffer -> écrit le buffer dans le fichier
bos.flush();

// close le stream, Close un buffer close aussi le stream
bos.close();

```

Exemple Input (pour lire sur un fichier) stream (**FOR BYTES[]**) :

```

InputStream fs = new FileInputStream("file.data");

// utiliser un buffer pour tous charger -> lire
BufferedInputStream bis = new BufferedInputStream(fs);

int b;
while ((b = bis.read()) != -1) {
    System.out.print(b);
}

// close le stream, Close un buffer close aussi le stream
bs.close();

```

C'est fonction peuvent retourner une **FileNotFoundException**

Pour les String

Y a une sous type de stream qui permette de géré les caractère facilement

```

Reader reader = new FileReader("file.data", StandardCharsets.UTF_8);

```

et le buffer

```

BufferedReader br = new BufferedReader(reader);

```

Exemple de lecture de string

```
String line;
while ((line = br.readLine()) != null) {
    // Careful: line does not contain end of line characters
    bw.write(line + END_OF_LINE);
}
```

```
Writer writer = new FileWriter("file.data", StandardCharsets.UTF_8);
```

et le buffer

```
BufferedWriter bw = new BufferedWriter(writer);
```

après ça s'utilise comme le stream de base

BREF, faux just foutre tous ça dans un **try/catch** pour choper les **IOException**



Cheatsheet

Build and tag an image

```
docker build -t <image-name> <build-context>
```

Start a container using its image name

```
docker run <image-name>
```

Start a container in background

```
docker run -d <image-name>
```

Display all running containers

```
docker ps
```

Stop a container

```
docker stop <container-id>
```

Access a running container

```
docker exec -it <container-id> /bin/sh
```

Create a Docker network

```
docker network create <name>
```

Start a container and override the entry point

```
docker run --entrypoint /bin/sh <image-name>
```

Start a container and override the command

```
docker run <image-name> <command>
```

```
docker run -d --rm -p --network <networkName> 8080:8080 plantuml/plantuml-server
```

-d = run **in** background

--rm = remove the container on stop

-p HOST:CONTAINER = map machine port **8080** to this container port **8080**

Delete all stopped containers

```
docker container prune
```

Delete all images

```
docker image prune
```

run compose

```
docker compose up -d
-d = run in background
```

```
# stop and remove compose
docker compose down
```

#IF YOU WANT TO exec, run, ps, ... -> JUST ADD 'compose' before the command

Create a docker file

```
# Image sur la quelle on se base pour créer la notre
FROM ubuntu:24.04
# Pour une app java FROM eclipse-temurin:21-jre

# Crée une variable d'environnement
ENV key=value

# copie un directory (Sur notre machine) dans un autre (dans l'image)
COPY sourceDir destDir

# en gros, c'est un cd dans l'image
WORKDIR path

# Exec command
CMD ["echo", "Hello, World!"]

# Expose port
EXPOSE 8000
```

Docker compose

OK, en gros, un docker compose c'est un fichier qui sert à créer une infra, genre le container c'est les machines et le compose il dit juste qu'il faut pour que ça marche et dans quel ordre les lancer

- service : Nom du service
- image : image à utiliser
- port : port à exposer (HOST:CONTAINER)
- volumes : mount volumes into the container
- environment : ENV variable
- network : network entre container

Example

```
networks:
  pantoufle:

services:
  ncat-server:
    hostname: my-server
    image: ncat
    command:
      - -l
      - "1234"
    networks:
      - pantoufle

  ncat-client:
    image: ncat
    command:
      - my-server
      - "1234"
    networks:
      - pantoufle
```

Protocol

Define

- Aperçus : Résumer du problème que dois résoudre le protocol
- Protocol utiliser : UDP / TCP ?
- Message : lists de messages du protocol
- Command : List d'action possible
- Example : Example de routine en UML

TCP

Client

```
// declare socket, writer, reader
try (Socket socket = new Socket(HOST, PORT);
    Reader reader = new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8);
    BufferedReader in = new BufferedReader(reader);
    Writer writer = new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8);
    BufferedWriter out = new BufferedWriter(writer); ) {
    // ...
} catch (IOException e) {
    // ...
}
```

Server

```
// declare server socket
try(ServerSocket serverSocket = new ServerSocket(PORT)){
    // ...
} catch (Exception e){
    // ...
}

// declare reader/writer
try (
    Socket socket = serverSocket.accept();
    Reader reader = new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8);
    BufferedReader in = new BufferedReader(reader);
    Writer writer = new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8);
    BufferedWriter out = new BufferedWriter(writer)) {
    // ...
} catch (IOException e) {
    // ...
}
```

UDP

Client

```
// declare socket
try (DatagramSocket socket = new DatagramSocket()) {
    // Get the server address
    InetAddress serverAddress = InetAddress.getByName(HOST);

    // Transform the message into a byte array - always specify the encoding
    byte[] buffer = MESSAGE.getBytes(StandardCharsets.UTF_8);

    // Create a packet with the message, the server address and the port
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length, serverAddress, PORT);

    // Send the packet
    socket.send(packet);

    System.out.println("[Client] Request sent: " + MESSAGE);
} catch (Exception e) {
    System.err.println("[Client] An error occurred: " + e.getMessage());
}
```

Server

```
// declare socket
try (DatagramSocket socket = new DatagramSocket(PORT)) {
    while (!socket.isClosed()) {
        // Create a buffer for the incoming request
        byte[] requestBuffer = new byte[1024];

        // Create a packet for the incoming request
        DatagramPacket requestPacket = new DatagramPacket(requestBuffer, requestBuffer.length);

        // Receive the packet - this is a blocking call
        socket.receive(requestPacket);

        // Transform the request into a string
        String request =
            new String(
                requestPacket.getData(),
                requestPacket.getOffset(),
                requestPacket.getLength(),
                StandardCharsets.UTF_8);
    }
} catch (Exception e) {
    // ...
}
```

cast

- Broadcast : Envoie a tous le monde ans le reseau

```
// add this
socket.setBroadcast(true);
// boardcast adress sois un subnet mask
InetAddress serverAddress = InetAddress.getByName("172.25.255.255");
```

- Multicast : Envoie a tous le monde qui correspond au masque
Sender

```
// Init multi cast address
InetAddress serverAddress = InetAddress.getByName("239.0.0.0");
```

Recever

```
// faux crée un MulticastSocket
try (MulticastSocket socket = new MulticastSocket(PORT)) {
    // Crée le group
    InetSocketAddress multicastGroup = new InetSocketAddress("239.0.0.0", PORT);
    // set un nom
    NetworkInterface networkInterface = NetworkInterface.getByName("NomRandom");
    // join le group au socket
    socket.joinGroup(multicastGroup, networkInterface);
}
```

Things -> Byte[]

STRING

```
String p = "I'm cooked for PCO :[";
byte[] data = p.getBytes(StandardCharsets.UTF_8);
```

Random class

```
// jsp si on a le droit mais au pire c'est là
public class IDK implements Serializable{
    ...
}
public static byte[] serialize(IDK obj){
    ByteArrayOutputStream bos = new ByteArrayOutputStream();
    try (ObjectOutputStream out = new ObjectOutputStream(bos)) {
        out.writeObject(obj);
        out.flush();
        return bos.toByteArray();
    } catch (Exception ex) {
        return new byte[0];
    }
}
public static IDK unserialize(byte[] data){
    ByteArrayInputStream bis = new ByteArrayInputStream(data);
    try (ObjectInputStream in = new ObjectInputStream(bis)) {
        return (IDK) in.readObject();
    } catch (Exception ex) {
        return null;
    }
}
```

ADD

- Read-eval-print loop (REPL) (jsp où le mettre)

Thread

Y a 2 manière

- les ExecutorService
- les thread (qu'on vas as utiliser parcequ'on est cringe)
(en vrais, c'est just qu'ils sont plus bare bonne, du coup ça vos pas le coup pour nous)

Usage

- Define "thread" function

```
public Integer SuperThreadFunc() {  
    // Manage emitters  
}
```

Version thread

```
public class ThreadClass implements Runnable{  
    public ThreadClass(/*args*/){  
        //...  
    }  
    @Override  
    public void run(){  
        //...  
    }  
}
```

- crée un Executor service

```
try (ExecutorService executorService = Executors.newFixedThreadPool(2); ) {  
    executorService.submit(this::SuperThreadFunc);  
    executorService.submit(new ThreadClass(/*...*/));  
    //...  
} catch (Exception e) {  
    System.out.println("[Receiver] Exception: " + e);  
  
    return 1;  
}
```

Special type

- y a les AtomicBoolean , AtomicInteger , ...
- et les ConcurrentHashMap , ConcurrentLinkedQueue

